CSA0419 - OPERATING SYSTEMS

ASSIGNMENT 1

Evaluate and Justify the Operating-System Design of a Smart Hospital

1. Understanding the assignment

Main scenario

A 500-bed smart hospital is developing a new computer system. It needs different types of computers and devices to perform different operations.

Hospital environment	Main purpose
Patient monitors	Monitor patient health and generate emergency alarms
Nurses' tablets	Access patient information while moving around
Doctors' desktops	View reports and manage patient records
Database server	Store and manage patient records
Imaging server	Process MRI scans using multiple processors
Partner hospitals	Share patient records between three hospitals

Main problem

The IT team wants to install one general-purpose operating system on every device. However, this creates several problems:

- Patient-monitor alarms may be delayed.
- The database server may stop during updates or failures.
- Tablets may consume too much battery.
- MRI processing requires high computing performance.
- Patient records must be protected from unauthorized access.

Main objective

To evaluate and select the most suitable operating-system type, structure, services and system calls for each hospital environment.

Important: You should not simply describe operating systems. You must compare alternatives, identify their advantages and disadvantages, and justify why your selected solution is better.

2. Explanation of all 12 tasks

TASK 1 – Evaluate the Requirements of Each Computing Environment (4 Marks)

What does the question ask?

Identify the requirements of every hospital device and rank them according to their importance.

You must discuss:

- Response time
- Reliability
- Throughput
- Battery consumption
- Security
- Patient safety

Answer

Different hospital environments require different operating-system features. Therefore, using one operating system for every device may not provide the best performance.

A. Ranked requirement table

Environment	**1st priority**	**2nd priority**	**3rd priority**
Patient monitors	Response time	Reliability	Security
Nurses' tablets	Battery life	Security	Response time
Doctors' desktops	Security	Performance	Reliability
Database server	Reliability	Security	Throughput
Imaging server	Processing speed	Throughput	Reliability
Partner hospitals	Security	Reliability	Communication

B. Explanation of requirements

1. Patient monitors

- Must detect abnormal readings and generate alarms within 10 milliseconds.
- Require a hard real-time operating system.
- Reliability is essential because delays can affect patient safety.
- Security prevents unauthorized modification of readings.

2. Nurses' handheld tablets

- Require low power consumption to operate throughout a shift.
- Need a handheld or mobile operating system.
- Must provide secure access to patient information.
- Should support wireless communication.

3. Doctors' desktop workstations

- Require a desktop operating system such as Windows or Linux.
- Need a graphical user interface.
- Must support medical applications and patient-record access.
- Require strong user authentication.

4. Database server

- Requires high reliability and continuous availability.
- Must support multiple users accessing patient records.
- Requires backup, security and fault recovery.
- A clustered operating system environment is suitable.

5. Imaging server

- Requires high processing power for MRI image processing.
- Must support multiple processors.
- Needs high memory capacity and fast storage.
- A multiprocessor operating system is suitable.

6. Partner hospitals

- Require secure communication and data sharing.
- Need a distributed operating-system environment.
- Must support authentication and access control.
- Reliable network communication is important.

C. Conflicting requirements

- Fast response time may conflict with heavy background processing.
- High throughput may increase competition for CPU resources.
- Strong security checks may introduce additional processing overhead.
- High performance and long battery life may conflict on handheld devices.

D. Application of operating-system history

OS development	**Hospital application**
Batch systems	Processing large medical reports
Time-sharing systems	Multiple doctors accessing the server
Personal computer systems	Doctors' desktops
Mobile systems	Nurses' tablets

E. Alternative concept – One general-purpose OS everywhere

If the hospital uses one general-purpose OS for every device:

- Alarm response time cannot necessarily be guaranteed because normal scheduling may delay execution.
- Each device may require more memory and storage than necessary.
- Unnecessary services increase resource consumption.
- System administration becomes complicated because the same OS must support different hardware requirements.

Evaluation: Different operating systems are better because each environment has different performance, safety and resource requirements.

TASK 2 – Judge the Type of Operating System for Each Environment (3 Marks)

What does the question ask?

Select the appropriate operating-system type for every hospital environment and justify your selection.

A. Comparison of operating-system types

OS type	Explanation	Hospital application
Mainframe OS	Supports large-scale processing and many users	Large-scale hospital data processing
Desktop OS	Designed for personal computers	Doctors' workstations
Multiprocessor OS	Uses multiple processors	MRI imaging server
Distributed OS	Supports connected computers and resource sharing	Partner hospitals
Clustered OS	Connects multiple computers for availability	Database server
Real-time OS	Provides predictable response within deadlines	Patient monitors
Handheld OS	Designed for portable, battery-powered devices	Nurses' tablets

B. Recommended OS for each environment

Environment	Recommended OS type	Reason
Patient monitors	Hard real-time OS	Guarantees critical alarm deadlines
Nurses' tablets	Handheld OS	Better battery and mobility support
Doctors' desktops	Desktop OS	Supports medical applications and GUI
Database server	Clustered OS	Provides fail-over and availability
Imaging server	Multiprocessor OS	Supports parallel processing

Partner hospitals	Distributed OS environment	Enables secure record sharing

C. Hard real-time vs soft real-time

- **Hard real-time OS:** Missing a deadline is unacceptable. Example: patient-monitor emergency alarms.
- **Soft real-time OS:** Missing an occasional deadline reduces performance but does not cause immediate system failure. Example: non-critical medical report processing.

D. Symmetric vs asymmetric multiprocessing

Symmetric multiprocessing (SMP)	**Asymmetric multiprocessing (AMP)**
All processors share system responsibilities.	Processors have assigned responsibilities.
Better workload distribution.	Workload is controlled by designated processors.
Suitable for parallel MRI processing.	Suitable when specific tasks need dedicated processors.

Recommendation: SMP is preferable for the imaging server because MRI processing can be distributed across multiple processors.

E. Clustered database server

A clustered system connects multiple servers to provide continuous database services.

- One server handles the main workload.
- Another server remains ready as a backup.
- If the primary server fails, the backup takes over.
- This reduces service interruption.

F. Alternative concept – One large multiprocessor server

If the hospital uses one large multiprocessor server instead of a cluster:

- It may provide high processing performance.
- It requires fewer separate machines.
- However, hardware failure can stop the entire database service.
- It creates a single point of failure.
- Adding processors can improve throughput, but performance does not increase indefinitely because of memory and communication bottlenecks.

Evaluation: A clustered server is preferable because availability and patient-record accessibility are more important than relying on one powerful machine.

TASK 3 – Evaluate Operating-System Structures (4 Marks)

What does the question ask?

Compare different operating-system structures and select the best structure for patient monitors and database servers.

A. Types of operating-system structures

Structure	Explanation	Advantages	Disadvantages
Monolithic	All major OS services operate inside one kernel	High performance	Failure in one component can affect the entire system
Layered	OS is divided into different layers	Easy to understand and maintain	Communication between layers can add overhead
Microkernel	Only essential functions remain in the kernel	Better reliability and isolation	Message passing may add overhead
Modular	Supports dynamically loadable kernel modules	Flexible and easy to extend	Faulty modules can affect kernel stability
Hybrid	Combines features of multiple structures	Balances performance and flexibility	More complex design

B. Recommended structures

1. Patient monitors – Microkernel structure

Reasons:

- Provides better isolation between services.
- Keeps the kernel small.
- Faulty device drivers can be isolated more effectively.
- Improves system reliability.
- Suitable for safety-critical real-time applications.

2. Database server – Modular or hybrid structure

Reasons:

- Supports high-performance processing.
- Allows additional modules and drivers.
- Provides flexibility for database operations.
- Supports efficient resource management.
- Suitable for large-scale server applications.

C. Examples of OS structures

Operating system	**General structure**
Linux	Monolithic kernel with loadable modules
Windows	Hybrid kernel
Android	Linux-based modular architecture
FreeRTOS	Small, real-time kernel
QNX	Microkernel architecture

D. Alternative concept – Monolithic kernel for patient monitors

If a monolithic kernel is used instead of a microkernel:

- System calls may execute with less communication overhead.
- Kernel services can communicate directly.
- The kernel may contain more functionality.
- A faulty driver running in kernel space can crash the entire operating system.
- Reliability and fault isolation may decrease.

Evaluation: A microkernel is preferable for patient monitors because reliability and fault isolation are more important than a small improvement in service-request speed.

TASK 4 – Evaluate the Services an Operating System Must Provide (4 Marks)

What does the question ask?

Explain the operating-system services required by each hospital environment. Also identify services that can be removed from small devices.

A. Important OS services

OS service	Explanation
User interface	Allows users to interact with the system
Program execution	Loads and executes programs
I/O operations	Controls input and output devices
File system	Organizes and manages files
Communication	Supports data exchange
Error detection	Identifies system errors
Resource allocation	Allocates CPU, memory and devices
Accounting	Records resource usage
Protection	Prevents unauthorized access
Security	Protects information and system resources

B. Services required for each environment

Environment	Important services
Patient monitors	Program execution, I/O, error detection, protection
Nurses' tablets	GUI, file management, communication, security
Doctors' desktops	GUI, file management, communication, protection
Database server	File management, resource allocation, security, error detection
Imaging server	Program execution, I/O, memory management, resource allocation

Partner hospitals	Communication, protection, authentication, security

C. Services that can be removed from patient monitors

To make patient monitors smaller and faster, unnecessary services can be removed.

- Advanced graphical interfaces.
- Unnecessary background applications.
- General-purpose file management features.
- Unused network services.
- Unnecessary multimedia services.

Essential services such as real-time scheduling, sensor I/O, error detection and protection must remain.

D. Essential database services

- File management for patient records.
- Memory management for database operations.
- Process and CPU scheduling.
- User authentication.
- Access control.
- Backup and recovery.
- Error detection.
- Network communication.

E. Patient-record protection

The operating system protects patient records using:

1. User authentication.
2. File permissions.
3. Access-control mechanisms.
4. Memory protection.
5. Process isolation.
6. Encryption and secure communication.

For example, a nurse should access only the records permitted by their role.

F. Alternative concept – Full desktop OS on tablets

If tablets use a full desktop operating system:

- More memory and storage are required.
- More background services consume CPU resources.
- Startup time may increase.
- Battery consumption may increase.
- Unnecessary services reduce efficiency.

Evaluation: A reduced handheld operating system is better because it provides the necessary hospital functions while saving memory, storage and battery power.

TASK 5 – Examine System Calls (4 Marks)

What does the question ask?

Explain how application programs communicate with the operating-system kernel using system calls.

A. Definition

A system call is a mechanism through which a user program requests services from the operating-system kernel.

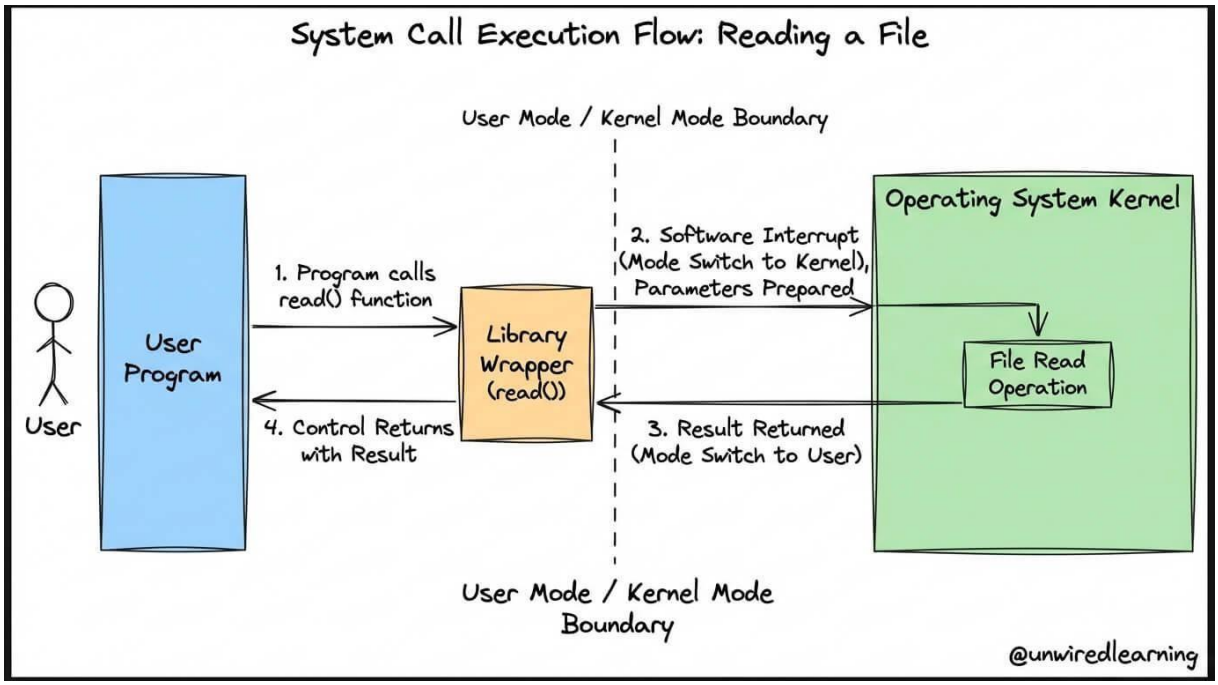
For example, when a program reads patient information from a file, it uses the read() system call.

B. Categories of system calls

Category	**Examples**	**Purpose**
Process control	fork(), exec(), exit()	Creates and controls processes
File management	open(), read(), write(), close()	Manages files
Device management	ioctl(), read(), write()	Controls devices
Information maintenance	getpid(), time()	Obtains system information
Communication	pipe(), socket()	Enables communication
Protection	chmod(), setuid()	Controls permissions

C. Steps involved in read() system call

1. A program calls the read() library function.
2. The library prepares the system-call number and parameters.
3. A trap or system-call instruction transfers control to the kernel.
4. The CPU switches from user mode to kernel mode.
5. The kernel checks the parameters and performs the requested operation.
6. The result is returned to the user program.
7. The CPU switches back to user mode.



unwiredlearning.com

D. Parameter-passing methods

- **Registers:** Parameters are stored in CPU registers.
- **Memory block:** Parameters are stored in a memory block and its address is passed.
- **Stack:** Parameters are placed on the program stack.

E. Using strace

strace is a Linux command-line tool that displays the system calls made by a program.

Example:

Bash

strace ./program

To count system calls:

Bash

strace -c ./program

F. Alternative concept – C library functions

Instead of directly using open(), read() and write(), programs can use:

- fopen()
- fread()
- fwrite()

These functions provide buffered file operations.

For a file containing $n$bytes:

- With a buffer of $b$bytes, approximately n/bread operations are needed.
- Library buffering can reduce system-call frequency.
- Extra memory is needed for the library buffer.

Evaluation: Buffered file operations are suitable for large backup files because they reduce the number of expensive system calls.

TASK 6 – Evaluate the Cost of System Calls (4 Marks)

What does the question ask?

Explain how system-call frequency affects program performance and execute the given file-copy program with different buffer sizes.

A. Why are system calls expensive?

A normal function call executes within the same program context. A system call requires a transition from user mode to kernel mode.

The additional work includes:

- Switching CPU execution modes.
- Validating system-call parameters.
- Executing kernel operations.
- Returning results to the application.

Therefore, a large number of system calls can reduce performance.

B. Effect of buffer size

A buffer temporarily stores data during file copying.

A larger buffer generally reduces the number of read and write system calls.

For a 10,000-byte file:

Buffer size	Expected read() calls
1 byte	10,001
512 bytes	21
4096 bytes	4

The extra call in each case is the final read() that detects the end of the file.

C. Complete the code – TODO 1

The following is a complete program based on the assignment's given code. The important part is the loop that replaces TODO 1.

C

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    if (bufsize == 0 || bufsize > sizeof(buf))
        return -1;
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) {
```

```
    close(in);
    return -1;
}
long calls = 0;
ssize_t n;
while (1) {
    n = read(in, buf, bufsize);
    calls++;
    if (n == 0)
        break;
    if (n < 0) {
        if (errno == EINTR) continue;
        calls = -1;
        break;
    }
    ssize_t written = 0;
    while (written < n) {
        ssize_t w = write(out, buf + written, n - written);
        if (w < 0) {
            if (errno == EINTR) continue;
            calls = -1;
            break;
        }
        if (w == 0) {
            calls = -1;
            break;
        }
        written += w;
    }
```

```
        if (calls == -1)
            break;
    }
    close(in);
    close(out);
    return calls;
}
int main(void)
{
    FILE *f = fopen("records.dat", "w");
    if (f == NULL) return 1;
    for (int i = 0; i < 10000; i++)
        fputc('A' + i % 26, f);
    fclose(f);
    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++) {
        printf("buffer %4zu bytes: %5ld read() calls\n",
               sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    }
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

D. How to execute in Linux

Save the program as copy.c.

Compile:

Bash

```
gcc -std=c11 -Wall -Wextra copy.c -o copy
```

Execute:

Bash

./copy

E. Expected output

Plain text

buffer 1 bytes: 10001 read() calls

buffer 512 bytes: 21 read() calls

buffer 4096 bytes: 4 read() calls

F. Run using strace

Bash

strace -c ./copy

This displays the system-call statistics.

Important: The program's own output counts read() calls made by copy_file(). The strace output includes other system calls made by the complete program, so the totals need not be identical.

G. Alternative concept – Memory-mapped files (mmap)

Instead of using read() and write(), the program can use mmap().

- The file is mapped into the process's virtual address space.
- The program accesses file data through memory.
- It reduces explicit read and write system calls.
- It can avoid an additional user-space copy through a separate read buffer.
- Very large files require careful memory management.

Evaluation: mmap() can be useful for large files and random access. Buffered read() and write() are simpler and provide more direct control over copying.

TASK 7 – Evaluate Process Control System Calls (4 Marks)

What does the question ask?

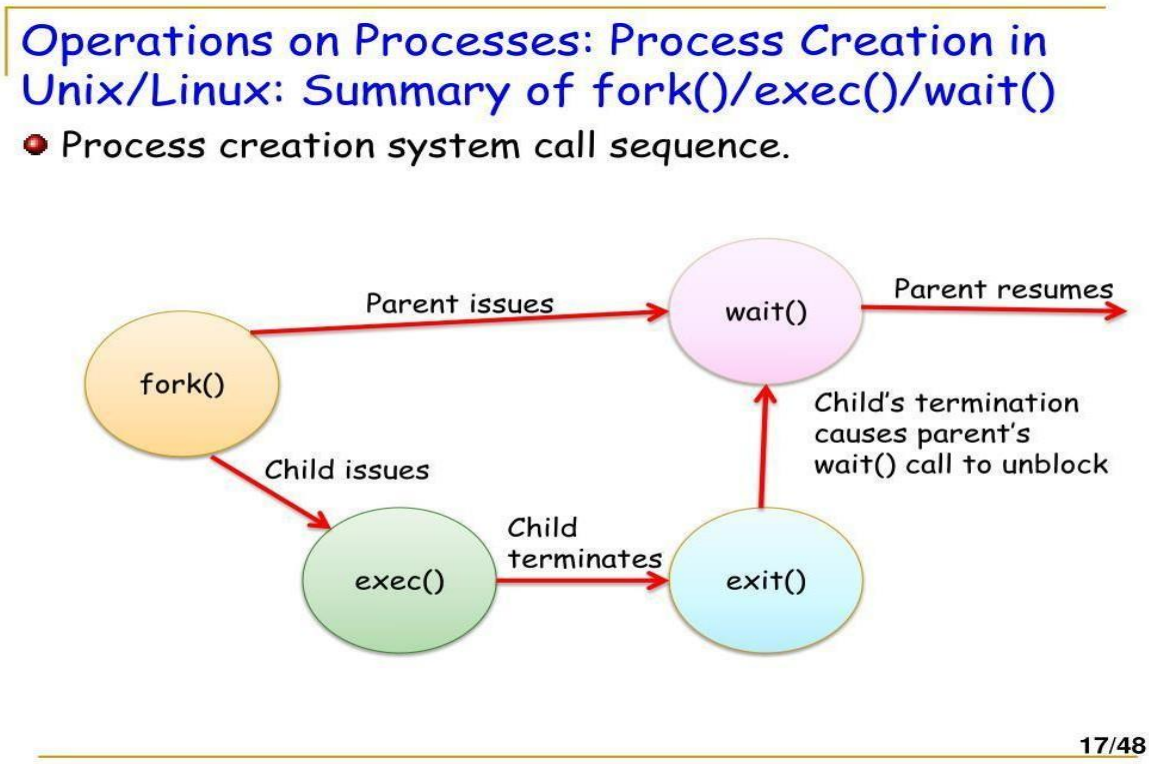
Explain how an operating-system shell creates, executes and manages processes.

A. Important process-control system calls

System call	Function
fork()	Creates a child process
exec()	Replaces the current process with a new program
wait()	Waits for a child process
exit()	Terminates a process
waitpid()	Waits for a specific child process

B. How a shell executes a command

1. The user enters a command.
2. The shell calls fork() to create a child.
3. The child calls exec() to execute the command.
4. The parent calls waitpid().
5. The child finishes and returns an exit status.
6. The parent collects the exit status.



slideserve.com

C. Zombie and orphan processes

Zombie process:

- A child process has completed execution.
- Its parent has not collected its exit status.
- The OS retains a small process-table entry until it is reaped.

Orphan process:

- A child continues running after its parent terminates.
- It is adopted by a suitable system process.
- The system process eventually collects its exit status.

D. Why fork() followed by exec()?

- fork() creates a separate child process.
- exec() loads and executes another program.
- The parent can continue independently.
- It provides process isolation and flexibility.

E. Complete the code – TODO 1

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);
    int status;
    pid_t pid = fork();
    if (pid < 0) {
```

```
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7",
                (char *)NULL);
        perror("exec");
        _exit(127);
    }
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("job 1 exit status %d\n",
                WEXITSTATUS(status));
    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0)
        _exit(3);
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("job 2 exit status %d\n",
                WEXITSTATUS(status));
    return 0;
}
```

F. Compile and execute

Bash

```
gcc -std=c11 -Wall -Wextra process.c -o process
```

Bash

```
./process
```

G. Expected output

Plain text

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

H. Why is stdout flushed before fork()?

When standard output is redirected to a file, it is generally buffered.

If fork() occurs before the buffer is flushed, both parent and child may inherit the same buffered data.

If the child uses exit(3), it flushes its inherited buffer, and the parent later flushes its own copy. Therefore, the message can appear twice.

_exit(3) terminates the child without flushing the inherited standard I/O buffer.

I. Alternative concept – posix_spawn() and threads

- posix_spawn() creates a new process and starts a program using a combined interface.
- Threads share the same process memory and resources.
- Processes provide stronger isolation.
- Threads generally require less memory and creation overhead.
- For independent shell commands, separate processes are safer.

Evaluation: fork() followed by exec() is suitable for a shell because it provides independent execution and process isolation.

TASK 8 – Evaluate CPU States and Protection (3 Marks)

What does the question ask?

Explain how the CPU protects operating-system resources and prevents unauthorized programs from affecting critical hospital devices.

A. User mode and kernel mode

User mode	**Kernel mode**
Runs normal applications	Runs operating-system kernel code
Has limited privileges	Has privileged access
Cannot execute privileged instructions	Can execute privileged instructions
Cannot directly access protected hardware	Can manage hardware resources

B. Privileged instructions and mode bit

- Privileged instructions are operations that only the kernel can execute.
- The mode bit identifies the current CPU execution mode.
- User applications cannot directly perform restricted hardware operations.
- The kernel controls access to important resources.

C. Interrupts, traps and timer interrupts

Interrupt: A signal that requests CPU attention, commonly from hardware.

Trap: A synchronous event caused by program execution, such as a system call or exception.

Timer interrupt: A periodic hardware interrupt that allows the OS to regain CPU control and manage scheduling.

D. What happens if a program disables interrupts?

- Patient-monitor tasks may not receive timely CPU attention.
- Other important processes may be delayed.
- System scheduling can be affected.
- Emergency alarms may miss their deadlines.

Therefore, disabling interrupts must be restricted to authorized kernel operations.

E. Memory protection

The OS and hardware use mechanisms such as:

- Memory Management Unit (MMU).
- Virtual memory.
- Page tables.

- Memory access permissions.
- Process isolation.

These mechanisms prevent one program from accessing another program's protected memory.

F. Alternative concept – Polling instead of interrupts

Polling: The CPU repeatedly checks whether a device has new data.

Interrupt-driven approach: The device notifies the CPU when an event occurs.

Polling	**Interrupts**
Repeatedly checks the sensor	Sensor notifies the CPU
May waste CPU time	Better CPU efficiency
Detection delay depends on polling interval	Detection depends on interrupt latency and scheduling
Can consume more power	Usually saves power when idle

For example, if polling happens every 20 milliseconds, an abnormal reading could wait nearly 20 milliseconds before being noticed.

Evaluation: Interrupt-driven I/O is preferable for patient monitors because it avoids unnecessary polling and can provide faster event notification. A suitable real-time system must still guarantee the complete 10-millisecond response deadline.

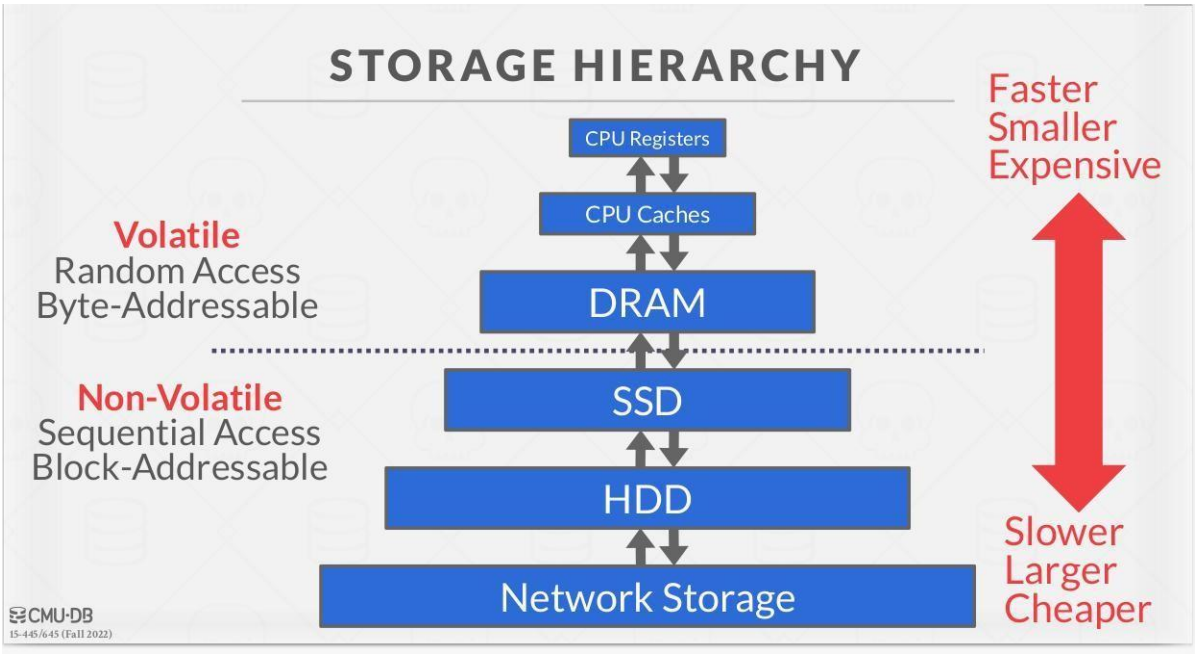
TASK 9 – Evaluate the Memory Hierarchy and I/O Systems (4 Marks)

What does the question ask?

Explain how hospital data is stored and transferred between different memory and storage devices.

A. Memory hierarchy

The memory hierarchy arranges storage according to speed, capacity and cost.



maxnilz.com

Memory level	**Speed**	**Capacity**	**Purpose**
Registers	Fastest	Very small	CPU operations
Cache	Very fast	Small	Frequently used data
Main memory (RAM)	Fast	Medium/Large	Running applications
SSD	Slower than RAM	Large	Patient records and applications
Hard disk	Slower	Very large	Backups and archives
Network storage	Depends on network	Very large	Shared hospital records

Access speeds depend on the hardware and configuration; the table shows relative performance rather than fixed timings.

B. Recommended data placement

Data	Recommended storage	Reason
MRI images	SSD and network storage	Large image files and fast retrieval
Patient records	Database storage with SSD	Fast access and reliable storage
Monitor readings	RAM buffers and persistent database	Quick processing and long-term storage
Backup data	Network storage or hard disk	Long-term recovery

C. Types of I/O

1. Programmed I/O

- CPU continuously controls data transfer.
- Simple but wastes CPU resources.
- Suitable for simple, low-volume operations.

2. Interrupt-driven I/O

- Device sends an interrupt when it needs attention.
- CPU handles the event.
- Suitable for event-based devices such as patient monitors.

3. Direct Memory Access (DMA)

- Transfers data directly between a device and main memory.
- Reduces CPU involvement.
- Suitable for large MRI image transfers.

D. Caching

Caching stores frequently accessed data in faster memory.

For example, frequently accessed patient records can be cached in RAM.

Benefits:

- Faster record retrieval.
- Reduced storage access.
- Improved application performance.

- Reduced response time for repeated queries.

E. Alternative concept – Interrupt-driven I/O instead of DMA

Suppose an MRI image contains 500 MB of data.

If interrupt-driven I/O generates one interrupt for every word, the number of interrupts depends on the word size.

For illustration, assuming 4-byte words:

$$\text{Number of interrupts} = \frac{500 \times 1024 \times 1024}{4}$$

$$= 131{,}072{,}000$$

This produces a very large number of interrupts.

Comparison:

Interrupt-driven I/O	**DMA**
CPU handles repeated interrupts	DMA controller manages bulk transfer
High CPU overhead	Lower CPU overhead
More processing involvement	Less CPU involvement
Suitable for smaller transfers	Suitable for large MRI images

Evaluation: DMA is the better choice for MRI image transfers because it reduces CPU overhead and supports efficient bulk data movement.

TASK 10 – Measure and Compare (4 Marks)

What does the question ask?

This task requires you to execute your programs on Linux and collect actual performance measurements.

You must measure:

1. Number of system calls for different buffer sizes.
2. Time spent in user mode and kernel mode.
3. Time required to create 1,000 processes.
4. Time required to create 1,000 threads.
5. Differences between expected and measured results.

A. Measure system calls

Execute:

Bash

strace -c ./copy

Record the number of read() and write() system calls.

Use this table in your report:

Buffer size	**Expected read() calls**	**Actual read() calls**
1 byte	10001	Fill after execution
512 bytes	21	Fill after execution
4096 bytes	4	Fill after execution

B. Measure execution time

Execute:

Bash

time ./copy

The command displays:

- Real time – Total elapsed execution time.
- User time – Time spent executing user-level code.
- System time – Time spent executing kernel operations.

C. Measure process creation

Create a file named process_time.c.

C

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
```

```
    for (int i = 0; i < 1000; i++) {
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork");
            return 1;
        }
        if (pid == 0)
            _exit(0);
        waitpid(pid, NULL, 0);
    }
    printf("1000 processes created successfully\n");
    return 0;
}
```

Compile and execute:

Bash

```
gcc process_time.c -o process_time
time ./process_time
```

D. Measure thread creation

Create thread_time.c.

C

```
#include <stdio.h>
#include <pthread.h>
void *task(void *arg)
{
    return NULL;
}
int main(void)
{
    pthread_t t[1000];
```

```
    for (int i = 0; i < 1000; i++) {
        if (pthread_create(&t[i], NULL, task, NULL) != 0) {
            printf("Thread creation failed\n");
            return 1;
        }
    }
    for (int i = 0; i < 1000; i++)
        pthread_join(t[i], NULL);
    printf("1000 threads created successfully\n");
    return 0;
}
```

Compile:

Bash

```
gcc -pthread thread_time.c -o thread_time
```

Execute:

Bash

```
time ./thread_time
```

E. Comparison of processes and threads

Processes	**Threads**
Have separate virtual address spaces	Share the same process address space
Higher memory overhead	Generally lower memory overhead
More isolation	Less isolation
Generally more expensive to create	Generally cheaper to create
Suitable for independent applications	Suitable for parallel tasks within an application

F. Measurement table

Fill this table with your actual Linux execution results.

Measurement	**Process**	**Thread**
Number created	1000	1000
Real time	Your result	Your result
User time	Your result	Your result
System time	Your result	Your result

G. Alternative concept – Threads instead of processes

If threads are created instead of processes:

- Creation time may decrease.
- Memory consumption may decrease.
- Threads share code, data and other process resources.
- Communication between threads is easier.
- A faulty thread can affect the entire process.

Evaluation: Threads are preferable for lightweight parallel operations, while processes are preferable when stronger isolation is required.

TASK 11 – Recommend and Defend the Operating-System Plan (4 Marks)

What does the question ask?

Combine the decisions made in Tasks 1–10 and recommend a complete operating-system design for the smart hospital.

A. Final recommended OS plan

Hospital environment	**OS type**	**OS structure**	**Main requirement**
Patient monitors	Hard real-time	Microkernel	Guaranteed alarm response
Nurses' tablets	Handheld	Mobile architecture	Battery efficiency

Doctors' desktops	Desktop	Hybrid/modular	User-friendly interface
Database server	Clustered	Modular/hybrid	High availability
Imaging server	Multiprocessor	Modular	Parallel processing
Partner hospitals	Distributed	Distributed services	Secure communication

B. Services and protection mechanisms

Patient monitors

- Real-time scheduling.
- Sensor input/output.
- Error detection.
- Process and memory protection.
- Interrupt-driven I/O.

Nurses' tablets

- Wireless communication.
- User authentication.
- File management.
- Battery management.
- Secure access control.

Doctors' desktops

- Graphical user interface.
- Application execution.
- File management.
- Authentication and permissions.

Database server

- Data management.
- Backup and recovery.
- Access control.

- Memory management.
- Cluster fail-over.

Imaging server

- Multiprocessor scheduling.
- High-speed memory.
- DMA-based image transfer.
- Parallel processing.

Partner hospitals

- Secure communication.
- Authentication.
- Encryption.
- Distributed data access.

C. Limitations and risks

- Real-time operating systems may require specialized development.
- Cluster configuration and maintenance can be expensive.
- Network failures can interrupt communication between hospitals.
- Security vulnerabilities can expose confidential patient records.
- Different operating systems increase administration complexity.

D. Suitability for three hospitals

The design can support three hospitals through a distributed architecture.

- Each hospital maintains its local devices and services.
- Hospitals securely share patient records.
- Centralized or shared database services support information exchange.
- Network redundancy improves communication reliability.
- Authentication controls access to shared records.

E. Alternative concept – Virtual machines on two large servers

Virtualization allows multiple virtual machines to run on the same physical hardware.

Advantages:

- Better hardware utilization.
- Easier resource allocation.
- Simplified management.
- Reduced physical server requirements.

Disadvantages:

- Virtualization introduces CPU and memory overhead.
- Resource competition can affect response time.
- Failure of a physical server may affect multiple virtual machines.
- Hard real-time performance may become difficult to guarantee.

Evaluation: Virtualization is suitable for desktop, database and general hospital applications. However, critical patient monitors should use dedicated real-time hardware rather than depending on shared virtualized resources.

F. Final recommendation

A combination of real-time, handheld, desktop, multiprocessor, clustered and distributed operating systems is the most suitable solution for the smart hospital.

This design provides:

- Fast emergency response.
- High database availability.
- Efficient MRI processing.
- Better battery management.
- Secure patient information.
- Scalability across three hospitals.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Highest-weighted task – 8 Marks

What does the question ask?

The hospital introduces two new requirements:

1. Remote patient monitoring from patients' homes.
2. Cloud-based patient records shared with partner hospitals.

You must evaluate your original design and decide whether to retain, modify or redesign it.

A. Impact Analysis (2 Marks)

Changes affecting the original design:

Component	Original design	New requirement
Patient monitoring	Hospital-based monitors	Hospital and home monitoring
Communication	Internal hospital network	Internet-based communication
Database	Hospital database server	Cloud-based records
Security	Local access protection	End-to-end secure communication and cloud access control
Hospital connection	Three connected hospitals	Three hospitals and remote home devices

Important changes:

- Home devices require internet connectivity.
- Data must travel securely over public networks.
- Cloud services require authentication and access control.
- Distributed communication becomes more important.
- Network failures must be considered.

B. Evaluation of the Original Design (2 Marks)

The original design is suitable for the hospital environment but requires modifications to support home monitoring and cloud records.

Advantages of the original design:

- Real-time monitoring at the hospital.
- Reliable database infrastructure.
- Secure access to patient information.
- Efficient processing of medical images.

Limitations:

- It was primarily designed for hospital-based devices.
- It does not fully address internet-based remote monitoring.
- Local databases alone cannot provide the required cloud sharing.
- Remote communication introduces additional network delays and security risks.

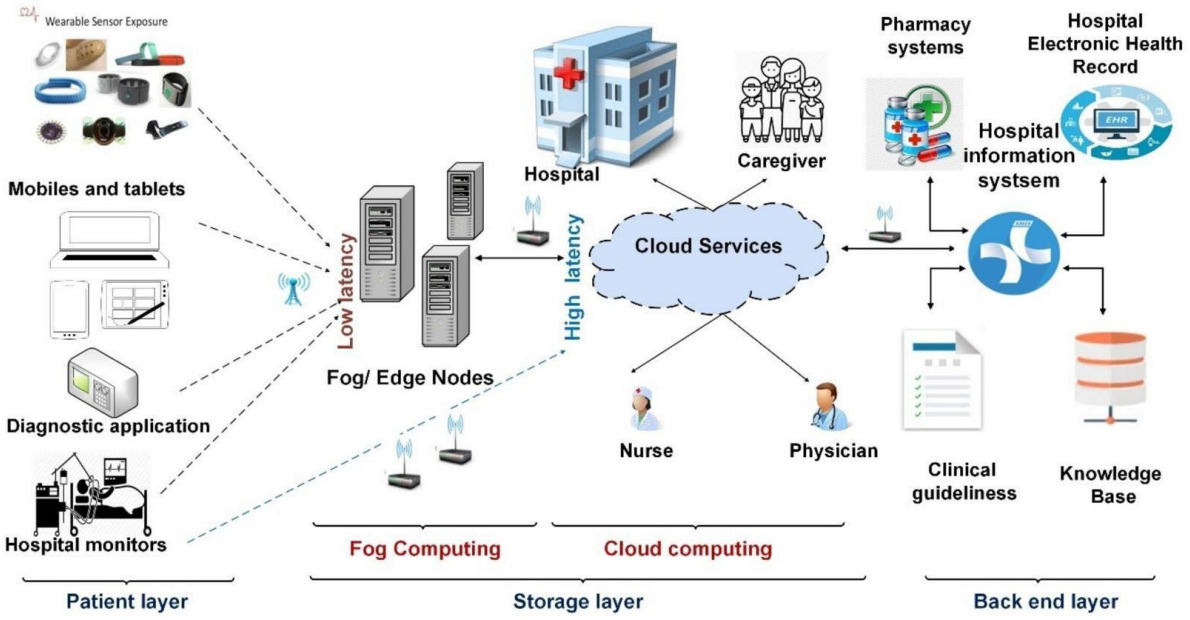
Evaluation: The original design cannot be retained without modifications.

C. Adaptation or Redesign (2 Marks)

The revised design should combine embedded real-time systems, distributed computing and cloud services.

Environment	**Revised solution**
Hospital monitors	Hard real-time OS
Home monitoring devices	Embedded real-time OS
Nurses' tablets	Handheld OS
Doctors' desktops	Desktop OS
Hospital imaging server	Multiprocessor OS
Patient records	Cloud-based database
Partner hospitals	Distributed system

Revised architecture



mdpi.com

Working process:

1. Home monitoring devices collect patient readings.
2. Embedded real-time operating systems process sensor information.
3. A secure communication connection sends readings to the hospital or cloud platform.
4. Cloud services store and process patient information.
5. Doctors access records through authenticated applications.
6. Partner hospitals securely access permitted shared records.
7. Emergency alerts are generated according to the monitoring system's safety requirements.

Important design consideration: Critical alarms should be handled locally or through a validated low-latency path. Internet and cloud communication should not be the only mechanism responsible for meeting a hard 10-millisecond alarm deadline.

D. Evidence-Based Defence (2 Marks)

The revised design is better because:

- Embedded real-time operating systems support predictable local monitoring.
- Cloud computing provides scalable storage.
- Distributed systems support sharing records between hospitals.
- Authentication and encryption improve data protection.

- Redundant communication improves reliability.
- Mobile operating systems support doctors and nurses accessing information remotely.

E. Final decision

Recommended decision: MODIFY

The original design should be modified rather than completely redesigned.

The existing hospital-based architecture remains useful. However, embedded home-monitoring devices, secure internet communication and cloud-based distributed records must be added.

Conclusion: The modified smart-hospital operating-system architecture provides better scalability, remote accessibility, reliability and security while retaining the original patient-safety requirements.